

AI Synthetic Media Detector

Complete Technical Blueprint, Visual Heatmap Pipeline, React Native Mobile UI & AI Handover Instructions

ARCHITECTURE: CLOUD-FIRST
HYBRID

MOBILE: REACT NATIVE /
EXPO

ML ENGINE: GRAD-CAM
PYTORCH

1. Executive Summary & AI Handover Context

This document constitutes the complete technical handover specification for the **AI Media Authenticity & Deepfake Detector Platform**. The architecture targets multi-platform accessibility (Mobile Native iOS/Android, Web PWA, and Browser Context Extensions) while anchoring detection heavy lifting in a **Cloud-First Dynamic Model Ensemble Engine** to guarantee maximum accuracy and rapid adaptability to newly released generative AI models (e.g., Sora, Flux, Midjourney v7).

Key Architectural Principles:

- **Zero Client Latency Lock:** Heavy multi-modal inference (Vision Transformers, 3D-CNNs, FFT Spectral Analysis, Grad-CAM generation) runs in serverless GPU worker clusters.
- **Cross-Platform Universal Client:** Shared UI component architecture across React Native (Mobile) and Next.js (Web/Extension UI).
- **Explainable AI Visual Diagnostics:** Every classification returns localized visual heatmaps (Grad-CAM) showing exact pixels/regions flagged for manipulation (e.g., pupil asymmetry, edge blending, hair artifacting).
- **Native OS Share Sheet Integration:** Mobile app receives images/videos directly from social apps (X, Instagram, WhatsApp) without manual downloads.

2. Visual Heatmap Generation Engine (PyTorch + Grad-CAM)

The visual heatmap module calculates model feature activations in the final convolutional / ViT block to construct a 2D spatial intensity map highlighting synthetic artifacts. The heatmap is blended with the original media as a Color-Mapped Jet overlay and returned as a Base64 string for direct mobile/web rendering.

Implementation Code: `heatmap_engine.py`

```

import cv2
import torch
import torch.nn as nn
import numpy as np
import base64
from PIL import Image
from io import BytesIO

class GradCAMHeatmapGenerator:
    def __init__(self, model: nn.Module, target_layer: nn.Module):
        self.model = model
        self.target_layer = target_layer
        self.gradients = None
        self.activations = None

        # Register hooks
        self.target_layer.register_forward_hook(self._save_activations)
        self.target_layer.register_full_backward_hook(self._save_gradients)

    def _save_activations(self, module, input, output):
        self.activations = output

    def _save_gradients(self, module, grad_input, grad_output):
        self.gradients = grad_output[0]

    def generate_heatmap(self, input_tensor: torch.Tensor, class_idx: int = None) -> np.ndarray:
        self.model.eval()
        output = self.model(input_tensor)

        if class_idx is None:
            class_idx = torch.argmax(output, dim=1).item()

        self.model.zero_grad()
        score = output[0, class_idx]
        score.backward()

        gradients = self.gradients.data.cpu().numpy()[0]
        activations = self.activations.data.cpu().numpy()[0]

        weights = np.mean(gradients, axis=(1, 2))
        cam = np.zeros(activations.shape[1:], dtype=np.float32)

        for i, w in enumerate(weights):
            cam += w * activations[i]

        cam = np.maximum(cam, 0)
        if np.max(cam) > 0:
            cam = cam / np.max(cam)
        return cam

    def overlay_heatmap(self, raw_image_bytes: bytes, cam_map: np.ndarray, alpha: float = 0.5) -> str:
        nparr = np.frombuffer(raw_image_bytes, np.uint8)
        img = cv2.imdecode(nparr, cv2.IMREAD_COLOR)
        h, w, _ = img.shape

        resized_cam = cv2.resize(cam_map, (w, h))
        heatmap = cv2.applyColorMap(np.uint8(255 * resized_cam), cv2.COLORMAP_JET)

```

```

        overlay = cv2.addWeighted(img, 1.0 - alpha, heatmap, alpha, 0)

        _, buffer = cv2.imencode('.jpg', overlay, [int(cv2.IMWRITE_JPEG_QUALITY), 90])
        b64_str = base64.b64encode(buffer).decode('utf-8')
        return f"data:image/jpeg;base64,{b64_str}"

def generate_visual_report(model, target_layer, image_bytes: bytes, input_tensor: torch.Tensor) -
> dict:
    generator = GradCAMHeatmapGenerator(model, target_layer)
    cam = generator.generate_heatmap(input_tensor)
    b64_heatmap = generator.overlay_heatmap(image_bytes, cam)
    return {
        "heatmap_base64": b64_heatmap,
        "max_artifact_intensity": float(np.max(cam))
    }

```

3. Mobile Client Interface (React Native / Expo)

The mobile client is built with React Native and Expo, supporting deep link sharing (Share Extension) so users can analyze media immediately from Twitter, Instagram, or Photos.


```

import React, { useState, useEffect } from 'react';
import {
  StyleSheet,
  View,
  Text,
  TouchableOpacity,
  Image,
  ScrollView,
  ActivityIndicator,
  SafeAreaView,
  Dimensions,
  Share
} from 'react-native';
import * as ImagePicker from 'expo-image-picker';

const { width } = Dimensions.get('window');

interface DiagnosticFinding {
  layer: string;
  status: 'passed' | 'failed' | 'warning';
  detail: string;
}

interface AnalysisResult {
  isAiGenerated: boolean;
  confidencePercentage: number;
  overallScore: number;
  heatmapUri?: string;
  findings: DiagnosticFinding[];
}

export default function AIDetectorScanScreen({ sharedMediaUri }: { sharedMediaUri?: string }) {
  const [selectedImage, setSelectedImage] = useState<string | null>(sharedMediaUri || null);
  const [isAnalyzing, setIsAnalyzing] = useState(false);
  const [analysis, setAnalysis] = useState<AnalysisResult | null>(null);
  const [showHeatmap, setShowHeatmap] = useState(true);

  useEffect(() => {
    if (sharedMediaUri) {
      setSelectedImage(sharedMediaUri);
      runDetection(sharedMediaUri);
    }
  }, [sharedMediaUri]);

  const pickImage = async () => {
    const permission = await ImagePicker.requestMediaLibraryPermissionsAsync();
    if (!permission.granted) {
      alert('Permission to access gallery is required!');
      return;
    }

    const result = await ImagePicker.launchImageLibraryAsync({
      mediaTypes: ImagePicker.MediaTypeOptions.Images,
      quality: 1,
    });

    if (!result.canceled && result.assets[0].uri) {
      const uri = result.assets[0].uri;

```

```

        setSelectedImage(uri);
        setAnalysis(null);
        runDetection(uri);
    }
};

const runDetection = async (imageUri: string) => {
    setIsAnalyzing(true);
    try {
        const formData = new FormData();
        formData.append('file', {
            uri: imageUri,
            name: 'scan_media.jpg',
            type: 'image/jpeg',
        } as any);

        const response = await fetch('https://api.yourdetector.com/v1/analyze/image', {
            method: 'POST',
            headers: { 'Content-Type': 'multipart/form-data' },
            body: formData,
        });

        const data = await response.json();

        setAnalysis({
            isAiGenerated: data.analysis.is_ai_generated,
            confidencePercentage: data.analysis.confidence_percentage,
            overallScore: data.analysis.confidence_percentage / 100,
            heatmapUri: data.analysis.heatmap_base64,
            findings: [
                { layer: 'Spatial Transformer', status: data.analysis.confidence_percentage > 50 ?
'failed' : 'passed', detail: 'High boundary warp anomalies around skin/background' },
                { layer: 'Frequency FFT Domain', status: 'failed', detail: 'Grid-like high frequency
spectral peak detected' },
                { layer: 'C2PA Metadata', status: 'warning', detail: 'No signed cryptographic
provenance manifest found' }
            ]
        });
    } catch (err) {
        console.error(err);
        alert('Analysis failed. Check server connection.');
```

} finally {
 setIsAnalyzing(false);
 }
};

const shareReport = async () => {
 if (!analysis) return;
 await Share.share({
 message: `[AI Detection Report] Verdict: \${analysis.isAiGenerated ? 'AI Generated' :
'Authentic Media'} (\${analysis.confidencePercentage}% Confidence). Analyzed via AI Media Detector
App.`
 });
};

return (
 <SafeAreaView style={styles.container}>
 <ScrollView contentContainerStyle={styles.scrollContent}>

```

    { /* Header */
    <View style={styles.header}>
      <Text style={styles.title}>AI Media Inspector</Text>
      <Text style={styles.subtitle}>Scan images & videos for synthetic manipulation</Text>
    </View>

    { /* Media Preview Box */
    <View style={styles.mediaContainer}>
      {selectedImage ? (
        <Image
          source={{ uri: (showHeatmap && analysis?.heatmapUri) ? analysis.heatmapUri :
selectedImage }}
          style={styles.previewImage}
          resizeMode="cover"
        />
      ) : (
        <View style={styles.placeholderBox}>
          <Text style={styles.placeholderText}>No Media Selected</Text>
          <Text style={styles.placeholderSub}>Tap below or share from any app</Text>
        </View>
      )}

      { /* Heatmap Toggle Badge */
      {analysis?.heatmapUri && (
        <TouchableOpacity
          style={styles.heatmapBadge}
          onPress={() => setShowHeatmap(!showHeatmap)}
        >
          <Text style={styles.heatmapBadgeText}>
            {showHeatmap ? '🔥 Heatmap: ON' : '📷 Original'}
          </Text>
        </TouchableOpacity>
      )}
      </View>

      { /* Action Button */
      <TouchableOpacity style={styles.selectButton} onPress={pickImage} disabled={isAnalyzing}>
        <Text style={styles.selectButtonText}>{selectedImage ? 'Choose Different Media' :
'Select Photo / Video'}</Text>
      </TouchableOpacity>

      { /* Loading Spinner */
      {isAnalyzing && (
        <View style={styles.loadingContainer}>
          <ActivityIndicator size="large" color="#2563eb" />
          <Text style={styles.loadingText}>Running Ensemble Detection Pipeline...</Text>
          <Text style={styles.loadingSub}>Testing FFT frequency, ViT artifacts, and C2PA
manifests</Text>
        </View>
      )}

      { /* Result Card */
      {analysis && !isAnalyzing && (
        <View style={[styles.resultCard, analysis.isAiGenerated ? styles.borderAi :
styles.borderReal]}>

          <View style={styles.verdictHeader}>
            <View>
              <Text style={styles.verdictLabel}>VERDICT</Text>

```

```

        <Text style={[[styles.verdictTitle, { color: analysis.isAiGenerated ? '#dc2626' :
'#16a34a' }]]}>
            {analysis.isAiGenerated ? 'AI Generated / Modified' : 'Likely Authentic Media'}
        </Text>
    </View>
    <Text style={[[styles.confidenceBadge, { backgroundColor: analysis.isAiGenerated ?
'#fef2f2' : '#f0fdf4' }]]}>
        {analysis.confidencePercentage}%
    </Text>
</View>

    { /* Confidence Progress Bar */ }
    <View style={styles.progressBackground}>
        <View style={
            styles.fillBar,
            { width: `${analysis.confidencePercentage}%`, backgroundColor:
analysis.isAiGenerated ? '#dc2626' : '#16a34a' }
        }> />
    </View>

    { /* Diagnostic Breakdown */ }
    <Text style={styles.breakdownHeader}>Diagnostic Evidence Chain</Text>
    {analysis.findings.map((item, idx) => (
        <View key={idx} style={styles.findingRow}>
            <View style={styles.findingHeader}>
                <Text style={styles.findingLayer}>{item.layer}</Text>
                <Text style={
                    styles.statusTag,
                    item.status === 'failed' ? styles.tagRed : item.status === 'passed' ?
styles.tagGreen : styles.tagYellow
                }>
                    {item.status.toUpperCase()}
                </Text>
            </View>
            <Text style={styles.findingDetail}>{item.detail}</Text>
        </View>
    )))

    { /* Share Report Button */ }
    <TouchableOpacity style={styles.shareButton} onPress={shareReport}>
        <Text style={styles.shareButtonText}>Export Forensic Summary</Text>
    </TouchableOpacity>

</View>
)}

</ScrollView>
</SafeAreaView>
);
}

const styles = StyleSheet.create({
    container: { flex: 1, backgroundColor: '#f8f9fa' },
    scrollContent: { padding: 16, paddingBottom: 40 },
    header: { marginBottom: 16 },
    title: { fontSize: 22, fontWeight: '800', color: '#0f172a' },
    subtitle: { fontSize: 13, color: '#64748b', marginTop: 2 },
    mediaContainer: {
        width: '100%',

```

```

height: width * 0.75,
backgroundColor: '#0f172a',
borderRadius: 12,
overflow: 'hidden',
position: 'relative',
justifyContent: 'center',
alignItems: 'center',
},
previewImage: { width: '100%', height: '100%' },
placeholderBox: { alignItems: 'center', padding: 20 },
placeholderText: { color: '#94a3b8', fontSize: 16, fontWeight: '600' },
placeholderSub: { color: '#64748b', fontSize: 12, marginTop: 4 },
heatmapBadge: {
  position: 'absolute',
  top: 12,
  right: 12,
  backgroundColor: 'rgba(15, 23, 42, 0.85)',
  paddingVertical: 6,
  paddingHorizontal: 10,
  borderRadius: 20,
  borderWidth: 1,
  borderColor: '#334155',
},
heatmapBadgeText: { color: 'ffffff', fontSize: 11, fontWeight: '700' },
selectButton: {
  backgroundColor: '#2563eb',
  paddingVertical: 14,
  borderRadius: 8,
  alignItems: 'center',
  marginTop: 14,
},
selectButtonText: { color: 'ffffff', fontWeight: '700', fontSize: 14 },
loadingContainer: { marginTop: 24, alignItems: 'center' },
loadingText: { marginTop: 12, color: '#0f172a', fontWeight: '600', fontSize: 14 },
loadingSub: { color: '#64748b', fontSize: 12, marginTop: 2 },
resultCard: {
  backgroundColor: 'ffffff',
  borderRadius: 12,
  padding: 16,
  marginTop: 20,
  borderWidth: 2,
  elevation: 3,
},
borderAi: { borderColor: '#fca5a5' },
borderReal: { borderColor: '#86efac' },
verdictHeader: { flexDirection: 'row', justifyContent: 'space-between', alignItems: 'center',
marginBottom: 10 },
verdictLabel: { fontSize: 10, fontWeight: '800', color: '#64748b', letterSpacing: 0.5 },
verdictTitle: { fontSize: 18, fontWeight: '800', marginTop: 2 },
confidenceBadge: { fontSize: 14, fontWeight: '800', paddingHorizontal: 10, paddingVertical: 4,
borderRadius: 6, color: '#0f172a' },
progressBackground: { height: 8, backgroundColor: '#f1f5f9', borderRadius: 4, overflow:
'hidden', marginBottom: 16 },
fillBar: { height: '100%', borderRadius: 4 },
breakdownHeader: { fontSize: 12, fontWeight: '700', color: '#0f172a', marginBottom: 8,
textTransform: 'uppercase' },
findingRow: { backgroundColor: '#f8fadc', padding: 10, borderRadius: 6, marginBottom: 8,
borderWidth: 1, borderColor: '#e2e8f0' },
findingHeader: { flexDirection: 'row', justifyContent: 'space-between', marginBottom: 4 },

```

```

findingLayer: { fontSize: 12, fontWeight: '700', color: '#1e293b' },
statusTag: { fontSize: 9, fontWeight: '800', paddingHorizontal: 6, paddingVertical: 2,
borderRadius: 4 },
tagRed: { backgroundColor: '#fee2e2', color: '#991b1b' },
tagGreen: { backgroundColor: '#dcfce7', color: '#166534' },
tagYellow: { backgroundColor: '#fef3c7', color: '#92400e' },
findingDetail: { fontSize: 11, color: '#475569' },
shareButton: { backgroundColor: '#0f172a', paddingVertical: 12, borderRadius: 8, alignItems:
'center', marginTop: 12 },
shareButtonText: { color: '#ffffff', fontSize: 13, fontWeight: '700' }
});

```

4. End-to-End AI Handover Instruction Prompt

To pass this project to another AI model instance for continued implementation, paste the following system prompt block directly into the target AI's context window.

Copy & Paste Handover Prompt for Receiving AI

System Prompt / Context Instruction for New AI Agent:

You are taking over lead engineering for the "AI Synthetic Media Authenticity Detector" platform.

PROJECT SUMMARY & STATUS:

- Architecture: Cloud-Centered Hybrid. Microservice API (FastAPI) handles ensemble model evaluation and Grad-CAM heatmap generation.
- Clients: React Native (Mobile iOS/Android + Expo Share Intent), Next.js (Web Dashboard), and Chrome Extension (Manifest V3 context menu inspector).
- Visual Heatmaps: Handled via PyTorch Grad-CAM hooks and OpenCV JET colormaps. Heatmaps are rendered directly over media in the client UI.

COMPLETED COMPONENTS:

1. Grad-CAM visual heatmap generator module (`heatmap\_engine.py`) - PyTorch + OpenCV.
2. Complete React Native Mobile UI (`ScanScreen.tsx`) with image selection, progress feedback, toggleable visual heatmap overlay, and evidence breakdown cards.
3. FastAPI endpoint interface and C2PA metadata inspection scaffolding.

YOUR TASK LIST FOR THIS SESSION:

1. Implement the Celery + Redis async worker pipeline (`tasks.py`) to process uploaded videos asynchronously (sampling at 5 FPS and compiling per-frame scores).
 2. Write the API endpoint for `/v1/analyze/url` to fetch remote media directly for the Chrome extension background service worker.
 3. Maintain clean TypeScript type safety and Python docstrings throughout.
- Please acknowledge receipt of this context and ask if you need any specific backend environment variables to proceed!